Python Programming

Duplicate File Removal Automation Using Python

Objective

Design and develop a Python automation script that periodically scans a specified directory, identifies duplicate files using file checksums, deletes the duplicate files, generates a detailed log file, and sends the log file through email.

General Development Guidelines

While developing the automation script, follow these rules:

1. Accept input through command-line arguments or an input file.
2. Do not display operational messages on the console. Store all messages, errors, and operation details in a log file.
3. Define a separate function for every separate task.
4. Handle all expected exceptions to make the script robust.
5. Validate all input values before performing any operation.
6. Store all user-defined functions in a separate user-defined module.
7. Write clean, modular, reusable, and properly documented code.
8. Use meaningful names for functions, variables, modules, and files.
9. Provide Help and Usage options for the script.

Problem Statement

Develop an automation script named:

`DuplicateFileRemoval.py`
The script should perform the following operations:

1. Accept Directory Name

Accept the absolute path of a directory from the user through the command line.

The specified directory may contain duplicate files.

The script should recursively scan all files available inside the specified directory and its subdirectories.

2. Identify Duplicate Files

Calculate the checksum of every file using an appropriate hashing algorithm such as MD5 or SHA-256.

Files having the same checksum should be considered duplicates.

For every group of duplicate files:

- Keep the first file.
- Delete all remaining duplicate copies.
- Store the complete path of every deleted duplicate file in the log file.

The script should not identify duplicate files only based on their names. Duplicate detection must be performed using file content and checksum values.

3. Create Log Directory

Create a directory named:

`Marvellous`
The directory should be created in the current working directory or at an appropriate predefined location.

If the directory already exists, the script should use the existing directory instead of generating an error.

4. Create Log File

Create a log file inside the `Marvellous` directory.

The name of the log file should contain the date and time at which the log file is created.

Example:

`DuplicateRemovalLog_20_07_2026_23_30_15.log`
The log file should contain:

- Starting time of directory scanning
- Completion time of directory scanning
- Name of the directory scanned

- Total number of files scanned
- Total number of duplicate files found
- Total number of duplicate files deleted
- Complete paths of all deleted duplicate files
- Checksum values of duplicate files
- Errors encountered during execution
- Email delivery status

All operational messages should be written to the log file instead of being displayed on the console.

5. Periodic Execution

Accept a time interval in minutes through the command line.

The duplicate-file removal operation should be performed repeatedly after the specified time interval.

For example, if the interval is 50, the directory should be scanned every 50 minutes.

The time interval must be:

- Numeric
- Greater than zero
- Expressed in minutes

The script should continue running until it is manually terminated.

6. Email Notification

Accept the receiver's email address through the command line.

After completing each duplicate-file removal operation:

- Send an email to the specified receiver.
- Attach the generated log file to the email.
- Include operation statistics in the email body.

Required Email Body

The email body must contain the following information:

```
Starting time of scanning:
Total number of files scanned:
Total number of duplicate files found:
Total number of duplicate files deleted:
Completion time of scanning:
Name of scanned directory:
```

Example email body:

```
Jay Ganesh,

The duplicate-file removal operation has been completed successfully.

Operation Statistics:

Starting time of scanning: 20 July 2026, 11:30:15 PM
Completion time of scanning: 20 July 2026, 11:30:27 PM
Directory scanned: E:/Data/Demo
Total number of files scanned: 125
Total number of duplicate files found: 14
Total number of duplicate files deleted: 14

Please find the detailed log file attached to this email.

Regards,
Marvellous Automation System
```

Command-Line Execution

The script should be executed using the following command:

```
python DuplicateFileRemoval.py E:/Data/Demo 50
marvellousinfosystem@gmail.com
```

Command-Line Arguments

Argument 1: Script Name

```
DuplicateFileRemoval.py
```

This is the name of the main Python automation script.

Argument 2: Directory Path

```
E:/Data/Demo
```

This is the absolute path of the directory that may contain duplicate files.

Argument 3: Time Interval

```
50
```
This is the time interval in minutes after which the duplicate-file removal operation should be repeated.

Argument 4: Receiver Email Address

```
marvellousinfosystem@gmail.com
```
This is the email address of the receiver who should receive the operation report and log file.

Help Option

The script should support the following Help option:

```
python DuplicateFileRemoval.py --help
```
or

```
python DuplicateFileRemoval.py -h
```
The Help option should provide information about:

- Purpose of the script
- Required command-line arguments
- Format of the command
- Description of every argument
- Usage examples

Example Help information:

```
Duplicate File Removal Automation

This script scans a directory, identifies duplicate files using
checksums,
deletes duplicate files, creates a log file, and sends the log file
through email.

Usage:
    python DuplicateFileRemoval.py <DirectoryPath>
<IntervalInMinutes> <ReceiverEmail>

Example:
    python DuplicateFileRemoval.py E:/Data/Demo 50
marvellousinfosystem@gmail.com
```

Usage Option

The script should support the following Usage option:

```
python DuplicateFileRemoval.py --usage
```
or

```
python DuplicateFileRemoval.py -u
```
Expected output:

```
Usage:
python DuplicateFileRemoval.py <AbsoluteDirectoryPath>
<TimeIntervalInMinutes> <ReceiverEmailAddress>
```

Required Validations

The script must validate the following conditions before starting the operation:

Directory Validation

- Check whether the directory path is provided.
- Check whether the path is absolute.
- Check whether the directory exists.
- Check whether the supplied path represents a directory.
- Check whether the application has permission to access the directory.

Time-Interval Validation

- Check whether the interval is provided.
- Check whether the interval contains only a valid numeric value.
- Check whether the interval is greater than zero.

Email Validation

- Check whether the email address is provided.
- Validate the basic format of the email address.
- Handle email authentication and connection failures.

Command-Line Validation

- Check the total number of command-line arguments.
- Display Help or Usage information when required.
- Handle invalid or unsupported command-line options.

File Validation

Before calculating a checksum or deleting a file:

- Check whether the file exists.
- Check whether the path represents a regular file.
- Check whether the file is readable.
- Check whether the file can be deleted.
- Handle permission-related errors.
- Handle files that are locked or currently in use.

Expected Algorithm

1. Read command-line arguments.
2. Check whether Help or Usage is requested.
3. Validate the total number of command-line arguments.
4. Validate the directory path.
5. Validate the time interval.
6. Validate the receiver's email address.
7. Create the `Marvellous` directory.
8. Start the scheduled operation.
9. Record the scanning start time.
10. Recursively scan all files.
11. Calculate the checksum of every file.
12. Group files according to checksum values.
13. Identify duplicate files.
14. Keep one original file from each duplicate group.

15. Delete the remaining duplicate files.
16. Record deleted file names in the log file.
17. Calculate operation statistics.
18. Record the completion time.
19. Generate the email body.
20. Attach the log file.
21. Send the email.
22. Repeat the operation after the specified interval.

README File Requirements

Create a file named:

`README.md`
The README file should contain the following sections:

1. Project Title

```
Duplicate File Removal Automation
```

2. Project Description

Explain that the script periodically scans a directory, detects duplicate files using checksums, deletes duplicate copies, creates a detailed log file, and sends the log file through email.

3. Features

Mention features such as:

- Recursive directory scanning
- Checksum-based duplicate detection
- Automatic duplicate-file deletion
- Timestamp-based log generation
- Periodic execution
- Email notification
- Log-file attachment
- Input validation

- Exception handling
- Modular programming

4. Requirements

Mention:

- Supported Python version
- Required Python libraries
- Internet connection for sending email
- Email application password or SMTP credentials

5. Project Structure

Explain the purpose of every Python module and directory.

6. Command-Line Options

Document all required arguments:

```
Directory path
Time interval in minutes
Receiver email address
```

7. Execution Command

```
python DuplicateFileRemoval.py E:/Data/Demo 50
marvellousinfosystem@gmail.com
```

8. Help Command

```
python DuplicateFileRemoval.py --help
```

9. Usage Command

```
python DuplicateFileRemoval.py --usage
```

10. Log-File Information

Explain where the logs are stored and how the log filename is generated.

11. Email Configuration

Explain how sender email credentials should be configured securely.

12. Important Notes

Mention that:

- Deleted files may not be recoverable.
- Testing should first be performed on a sample directory.
- Email passwords should not be hard-coded.
- The first file from each duplicate group should be preserved.
- Files should be considered duplicates only when their checksums are identical.

Expected Output

After every scheduled execution:

1. Duplicate files should be removed from the supplied directory.
2. One original file from every duplicate group should remain.
3. A timestamp-based log file should be created inside the `Marvellous` directory.
4. The log file should contain the details of all deleted files.
5. Operation statistics should be recorded.
6. The log file should be sent to the receiver through email.
7. The operation should repeat after the specified interval.